Chapter 17 — Multipart & Resumable Uploads

> **Part II — Networking Patterns** · Complexity: **Standard** (≥25 interview questions)
>
> **Prerequisites:** Ch 1 (Networking Core), Ch 2 (retry/backoff), Ch 15 (idempotency), Ch 30 (WorkManager — background work; previewed here), Ch 62 (content providers/SAF — previewed). Pairs with Ch 18 (downloads).
>
> **Reference surface in StreamCart:** uploading a profile photo (small, multipart), a product review video, and a creator's long upload (large, resumable, background).

17.1 Problem Definition and Real-World Context

Uploading a file from a phone is far harder than a normal POST: files are large, networks drop mid-transfer, the app gets backgrounded or killed, and the user expects progress and the ability to leave the screen. Naive uploads fail:

- **Whole-file retry wastes everything.** A 200 MB video upload fails at 95% on a flaky connection; a naive client restarts from 0 — repeatedly — never finishing and burning the user's data.
- **Upload dies when the screen closes.** If the upload runs in a ViewModel/ Activity scope, navigating away or process death kills it; the user must stay on the screen staring at a spinner.
- **OOM from reading the whole file into memory.** Loading a large file into a byte array to build the request body crashes.
- **Duplicate uploads on retry.** Retrying a create-resource POST without idempotency (Ch 15) creates duplicates.
- **No progress / can't pause.** Users abandon uploads with no feedback.

The solutions scale with file size: **multipart form upload** (streamed, not buffered) for small files; **resumable/chunked upload** (upload in parts, resume from the last confirmed offset) for large files; **WorkManager + a foreground service** so uploads survive navigation/process death and continue in the background; plus **progress**, **retry/backoff** (Ch 2), **idempotency** (Ch 15), and **network constraints**.

Three practical use cases

1. **Profile photo (StreamCart Account).** Small image → streamed multipart with progress; retry on failure; idempotency key so a retry doesn't create two avatars.
2. **Review video (Shop/Social).** Large file → resumable upload in chunks, background via WorkManager + foreground notification, resumes after connection drop or app restart.
3. **Creator long upload (Watch/Social).** Very large → chunked + resumable, constrained to unmetered/charging, with pause/resume and durable progress.

Why this is architectural

Uploads define **where the work runs** (background-safe vs UI-scoped), **the transfer protocol** (whole vs chunked/resumable), **durability of progress**, **retry/idempotency**, and **resource streaming** — system properties that decide whether large uploads ever complete on mobile.

17.2 Requirements Engineering

Functional requirements

ID	Requirement
F1	Upload small files via streamed multipart with progress.
F2	Upload large files resumably (chunked), resuming from the last confirmed offset.
F3	Continue uploads across navigation and process death (WorkManager/foreground service).
F4	Report progress (%) to the UI; support cancel and (ideally) pause/resume.
F5	Retry transient failures with backoff; don't restart from 0 (resumable).
F6	Avoid duplicate server resources on retry (idempotency, Ch 15).
F7	Stream from content (no whole-file in memory); respect SAF/content URIs.
F8	Honor network constraints (Wi-Fi only / charging) where required.

Non-functional requirements

Attribute	Policy
Reliability	Large uploads complete over flaky networks (resume, not restart).
Memory	Stream file content; never buffer whole file.
Battery/Network	Constraints (unmetered/charging); backoff; chunk sizing.
Durability	Progress/offset persisted; survives process death.
UX	Live progress; background continuation; cancel/pause.
Security	Auth on upload; validate on server; no sensitive data leakage.

Scope

- **In scope:** streamed multipart, resumable/chunked protocol, progress, WorkManager + foreground service, retry/idempotency integration, content-URI streaming, network constraints.
- **Out of scope:** WorkManager internals (Ch 30), foreground-service details (Ch 46), SAF/content providers (Ch 62), media processing/transcoding, download (Ch 18).

Assumptions & constraints

- OkHttp/Retrofit; backend supports multipart and a resumable protocol (tus-like or Google-resumable: `init → PUT chunks with Content-Range → status`). WorkManager for background. `minSdk 24`.

17.3 System Design Fundamentals

- **Streamed vs buffered body.** Build the request body as a stream over the file's `InputStream` (`RequestBody` backed by the content URI) so memory stays O(chunk), not O(file). Buffering a large file into a `ByteArray` OOMs.
- **Multipart** (`multipart/form-data`): the standard for a single-shot file + fields; fine for small/medium files; a failure restarts the whole part.
- **Resumable/chunked upload.** The client first **initializes** an upload session (gets an upload URL/id), then uploads **chunks** with a `Content-Range`/offset; on failure it **queries the server for the last received offset** and resumes from there. This is the key to completing large uploads on mobile.
- **Background execution.** Uploads must outlive the UI. **WorkManager** (Ch 30) schedules a worker (with a **foreground service**, Ch 46, for long/expedited work) so the upload continues across navigation and process death, honoring **constraints** (network type, charging).
- **Durable progress/offset.** Persist the session id + last confirmed offset (Room/DataStore) so a restart resumes rather than restarts.
- **Idempotency (Ch 15).** The *initialize* step (which creates the server resource) carries an idempotency key so a retry doesn't create duplicates; chunk PUTs are naturally idempotent (same range).
- **Progress reporting.** Wrap the request body to emit bytes-written; surface via WorkManager progress / a Flow to the UI.

Core insight: **size determines strategy** — stream a multipart for small files; for large files, upload resumably in chunks with a durable offset, run it in a WorkManager foreground worker so it survives the UI and process death, and make the create step idempotent.

17.4 High-Level Architecture


17.4 High-Level Architecture

Components & ownership

- **`UploadWorker`** (WorkManager, foreground) owns the transfer; survives the UI.
- **Resumable session state** (uploadId + offset) persisted in Room.
- **Streamed `RequestBody`** reads the content URI chunk-by-chunk (O(chunk) memory) and reports progress.
- **Backend** provides init + chunked PUT + status (last offset) endpoints.
- **UI** observes `WorkInfo` progress; it does **not** run the upload.

Direction: UI enqueues → worker inits (idempotent) → streams chunks → persists offset → resumes on failure → reports progress to UI.

17.5 Detailed Component Design



Component	Responsibility	Notes
`UploadSession` (Room)	durable transfer state	uploadId + offset + idempotency key
`UploadWorker`	run transfer in background	foreground service; constraints; retry via `Result.retry()`
`ProgressRequestBody`	stream + report progress	O(chunk) memory
`ResumableUploadApi`	init/chunk/status	server protocol

17.6 End-to-End Data Flow

Resumable upload with a connection drop



Small multipart (streamed)



17.7 Complete Kotlin Implementation

Package: `com.streamcart.upload`.

17.7.1 Streaming, progress-reporting request body (no whole-file in memory)

```
class ProgressRequestBody(
    private val contentResolver: ContentResolver,
    private val uri: Uri,
    private val contentType: String,
    private val rangeStart: Long = 0,
    private val rangeEnd: Long = Long.MAX_VALUE,          // for chunked uploads
    private val onProgress: (bytesSent: Long) -> Unit,
) : RequestBody() {
    override fun contentType() = contentType.toMediaTypeOrNull()
    override fun contentLength(): Long = (rangeEnd - rangeStart).coerceAtLeast(-1)

    override fun writeTo(sink: BufferedSink) {
        contentResolver.openInputStream(uri)!!.use { input ->
            input.skip(rangeStart)
            val buffer = ByteArray(DEFAULT_BUFFER_SIZE)    // O(chunk) memory
            var sent = rangeStart
            while (sent < rangeEnd) {
                val toRead = minOf(buffer.size.toLong(), rangeEnd - sent).toInt()
                val read = input.read(buffer, 0, toRead)
                if (read == -1) break
                sink.write(buffer, 0, read)
                sent += read
                onProgress(sent)
            }
        }
    }
}
```

17.7.2 Resumable API

```
interface ResumableUploadApi {
    @POST("uploads")
    suspend fun init(@Header("Idempotency-Key") key: String, @Body meta: UploadMeta): InitResult

    @PUT("uploads/{id}")
    suspend fun uploadChunk(
        @Path("id") uploadId: String,
        @Header("Content-Range") contentRange: String,     // "bytes start-end/total"
        @Body body: RequestBody,
    ): Response<UploadResult>

    @GET("uploads/{id}/status")
    suspend fun status(@Path("id") uploadId: String): StatusResult   // last received offset
}
```

17.7.3 UploadWorker (background, resumable, foreground)

```
@HiltWorker
class UploadWorker @AssistedInject constructor(
    @Assisted ctx: Context,
    @Assisted params: WorkerParameters,
    private val api: ResumableUploadApi,
    private val dao: UploadSessionDao,
    private val resolver: ContentResolver,
) : CoroutineWorker(ctx, params) {

    override suspend fun getForegroundInfo() =
        ForegroundInfo(NOTIF_ID, buildUploadNotification(progress = 0))   // Ch 46

    override suspend fun doWork(): Result {
        val localId = inputData.getString(KEY_LOCAL_ID) ?: return Result.failure()
        val session = dao.get(localId) ?: return Result.failure()
        setForeground(getForegroundInfo())

        return try {
            // 1) Init once (idempotent) — reuse uploadId on resume.
            val uploadId = session.uploadId ?: run {
                val id = api.init(session.idempotencyKey, session.meta()).uploadId
                dao.update(session.copy(uploadId = id)); id
            }

            // 2) Resume from the server's last-known offset (source of truth).
            var offset = api.status(uploadId).receivedBytes
            dao.update(session.copy(bytesSent = offset))

            // 3) Upload remaining in chunks.
            while (offset < session.totalBytes) {
                val end = minOf(offset + CHUNK, session.totalBytes)
                val body = ProgressRequestBody(
                    resolver, Uri.parse(session.contentUri), session.mime,
                    rangeStart = offset, rangeEnd = end,
                ) { sent -> /* throttle */ }
                val range = "bytes $offset-${end - 1}/${session.totalBytes}"
                val resp = api.uploadChunk(uploadId, range, body)
                if (!resp.isSuccessful && resp.code() != 308) return Result.retry()  //
transient -> backoff
                offset = end
                dao.update(session.copy(bytesSent = offset))
                setProgress(workDataOf(KEY_PROGRESS to (offset * 100 /
session.totalBytes).toInt()))
```

```
                setForeground(ForegroundInfo(NOTIF_ID, buildUploadNotification((offset * 100 /
session.totalBytes).toInt())))
            }
            dao.update(session.copy(status = Status.COMPLETE))
            Result.success()
        } catch (e: IOException) {
            Result.retry()                                  // WorkManager backoff +
constraints
        }
    }
    private companion object { const val CHUNK = 5L * 1024 * 1024; const val KEY_LOCAL_ID =
"local_id"; const val KEY_PROGRESS = "progress" }
}
```

17.7.4 Enqueue with constraints + unique work

```
fun enqueueUpload(context: Context, localId: String, requireUnmetered: Boolean) {
    val request = OneTimeWorkRequestBuilder<UploadWorker>()
        .setInputData(workDataOf("local_id" to localId))
        .setConstraints(
            Constraints.Builder()
                .setRequiredNetworkType(if (requireUnmetered) NetworkType.UNMETERED else
NetworkType.CONNECTED)
                .build()
        )
        .setBackoffCriteria(BackoffPolicy.EXPONENTIAL, 10, TimeUnit.SECONDS)   // Ch 2 analogue
        .build()
    WorkManager.getInstance(context)
        .enqueueUniqueWork("upload-$localId", ExistingWorkPolicy.KEEP, request) // dedup (Ch 30)
}
```

17.7.5 Small multipart upload (for small files)

```
interface AvatarApi {
    @Multipart @POST("profile/avatar")
    suspend fun upload(
        @Header("Idempotency-Key") key: String,
        @Part file: MultipartBody.Part,
    ): AvatarResult
}
// part built from a streamed ProgressRequestBody, not a byte array.
val part = MultipartBody.Part.createFormData(
    "file", "avatar.jpg",
    ProgressRequestBody(resolver, uri, "image/jpeg") { /* progress */ },
)
```

17.7.6 UI observes progress (not the transfer)

```
val progress: Flow<Int> = WorkManager.getInstance(context)
    .getWorkInfosForUniqueWorkFlow("upload-$localId")
    .map { it.firstOrNull()?.progress?.getInt("progress", 0) ?: 0 }
```

17.7.7 Module structure

```
:feature:upload       UploadWorker, enqueueUpload, progress UI
:data:upload          UploadSession(Dao), ResumableUploadApi, ProgressRequestBody
:core:notifications   upload notification (Ch 46/49)
```

17.8 Concurrency and Race Conditions

Scenario	Risk	Handling
Duplicate upload enqueued	two transfers of one file	`enqueueUniqueWork(KEEP)` (Ch 30)
Init retried	duplicate server resource	idempotency key on init (Ch 15)
Resume vs stale local offset	re-send or skip bytes	trust server `status` offset as source of truth
Process death mid-chunk	lost progress	persist offset after each confirmed chunk
Concurrent chunk PUTs	out-of-order ranges	upload sequentially (or server-ordered ranges)
Progress spam	UI/IPC overload	throttle `setProgress` updates

- **Server offset is the source of truth for resume:** on restart/failure, query `status(uploadId)` and continue from there — local counters can be stale.

- **Idempotent init + unique work** together prevent duplicate resources and duplicate transfers.

17.9 Failure Scenarios and Recovery

Failure	Detection	Response	UX
Connection drop mid-upload	`IOException`/non-2xx	`Result.retry()` + resume from offset	progress holds, resumes
App killed / process death	worker rescheduled	WorkManager restarts; resume from persisted/server offset	continues
Navigate away	—	runs in worker/foreground service	unaffected
5xx / timeout	transient	retry with backoff (WorkManager)	auto
4xx (bad request/auth)	non-retryable	fail; surface error; re-auth (Ch 19)	actionable error
Disk/permission to read URI lost	`SecurityException`	fail with message; re-pick file	clear error
Duplicate on retry	server resource dup	idempotent init prevents it	one resource
Metered network when unmetered required	constraint	worker waits for Wi-Fi	deferred



> **Resume, don't restart:** every confirmed chunk persists an offset; recovery continues from the server's last-received byte, so a 95%-complete upload never restarts from 0.

17.10 Scaling the Implementation

- **Level 1.** Streamed multipart in a coroutine with progress (small files, foreground).
- **Level 2.** Move to WorkManager so it survives navigation; add retry/backoff + constraints.
- **Level 3 — production.** Resumable chunked protocol with durable offset + foreground service; idempotent init; progress via `WorkInfo`; cancel.
- **Level 4 — large.** Pause/resume; parallel chunking (where server supports); adaptive chunk size by network quality (Ch 91); upload queue for multiple files.
- **Level 5 — enterprise.** Direct-to-storage (signed URLs to S3/GCS), client-side encryption, integrity (checksums), background upload scheduling policies, upload analytics/SLAs.

Avoid overengineering: a small avatar doesn't need chunking/resumable — stream a multipart. Reserve resumable + WorkManager + foreground service for large files that must survive drops and backgrounding.

17.11 Alternative Approaches and Trade-offs

Dimension	Option A	Option B	When
Protocol	Multipart (whole)	Resumable (chunked)	Multipart for small; resumable for large/flaky
Where it runs	WorkManager worker	UI/ViewModel coroutine	WorkManager for survive-nav/process-death; UI scope only for tiny quick uploads
Long upload	Foreground service	Expedited/normal worker	Foreground service for long user-visible uploads (Ch 46); expedited for short
Body	Streamed `RequestBody`	Buffered `ByteArray`	Always stream large files (memory)
Destination	Via your backend	Direct to object storage (signed URL)	Direct-to-storage offloads bandwidth; via-backend for processing/validation
Chunk size	Fixed (e.g., 5 MB)	Adaptive by network	Adaptive on variable networks; fixed for simplicity
Dedup	Idempotent init + unique work	none	Always for create operations

Multipart vs resumable: multipart is simpler (one request) but any failure restarts the whole transfer — unacceptable for large files on mobile. Resumable adds protocol complexity (init/chunk/status, offset tracking) but is the only way large uploads reliably complete. Choose by size and failure cost.

Via-backend vs direct-to-storage: uploading through your backend lets it validate/transcode but doubles bandwidth and load; signed-URL direct upload to object storage scales better but needs the backend to issue URLs and post-process. Large-scale media apps use direct-to-storage.

17.12 Security Considerations

- **Authenticate uploads** (Ch 19) and **authorize server-side**; a signed upload URL must be short-lived and scoped.
- **Validate on the server** (type, size, content) — never trust client-declared MIME/size; scan for malware; re-encode images to strip metadata/exploits.
- **Content-URI permissions:** use the SAF-granted URI (Ch 62); don't copy sensitive files unnecessarily; respect scoped storage.
- **Sensitive media:** consider client-side encryption before upload; strip EXIF/GPS from photos unless intended.
- **Idempotency key** prevents duplicate resources but is **not** auth (Ch 15).
- Client vs backend: the client streams and resumes; the backend authenticates, authorizes, validates, and owns the final resource. A tampered client must not be able to upload on another user's behalf.

17.13 Performance and Resource Optimization

- **Stream, don't buffer:** O(chunk) memory via `ProgressRequestBody` over the content URI — avoids OOM on large files.

- **Chunk size:** balance overhead (too small = many requests) vs retry cost (too big = more to re-send on failure); ~1–8 MB is typical; adapt to network (Ch 91).
- **Constraints:** unmetered/charging for large uploads saves user data/battery (Ch 30/47).
- **Resume saves bandwidth/battery:** no restart-from-0 on flaky links.
- **Throttle progress updates** to avoid IPC/recomposition overhead.
- **Profiling:** Network Inspector (chunk requests), battery/data stats, upload success rate by network type.

17.14 Testing Strategy

Test chunking/resume logic with MockWebServer; worker logic with WorkManager test utilities.

```
class ResumableUploadTest {
    private val server = MockWebServer()

    @Test fun `resumes from server offset after failure`() = runTest {
        // init -> uploadId; first chunk fails; status reports received=5MB; resume from 5MB
        server.enqueue(MockResponse().setBody("""{"uploadId":"U"}"""))
        server.enqueue(MockResponse().setResponseCode(503))                  // chunk fails
        server.enqueue(MockResponse().setBody("""{"receivedBytes":5242880}"""))
        server.enqueue(MockResponse().setResponseCode(201).setBody("""{"id":"r1"}"""))
        val result = runWorker(localId = seedSession(total = 10_485_760))
        // assert it continued from 5MB, not 0
        assertTrue(result is ListenableWorker.Result.Success || result is
ListenableWorker.Result.Retry)
    }

    @Test fun `streams without loading whole file`() {
        val body = ProgressRequestBody(resolver, uri10MB, "video/mp4") {}
        assertEquals(10_485_760, body.contentLength())   // known length, streamed in writeTo
    }

    @Test fun `init uses idempotency key, reused on resume`() = runTest {
        // seed session with a key; run twice; assert same Idempotency-Key header
    }

    @Test fun `unique work prevents duplicate uploads`() {
        enqueueUpload(ctx, "f1", false); enqueueUpload(ctx, "f1", false)
        assertEquals(1, WorkManager.getInstance(ctx).getWorkInfosForUniqueWork("upload-
f1").get().size)
    }
}
```

Also: `WorkManagerTestInitHelper` / `TestListenableWorkerBuilder` for the worker; progress emission; cancel; constraint gating; non-retryable 4xx fails.

17.15 Production Observability

- **Metrics:** upload success rate (by size/network), median/p95 upload time, resume rate (uploads that resumed), bytes re-sent (should be low with resumable), failure reasons, foreground-worker completion rate.
- **Logs:** `upload{id, size, network, offset, result}`, `upload_resume{from}`, `upload_error{type, code}`.
- **Debugging:** uploads never finishing on flaky networks → not resumable (restarting); OOM → buffering whole file; duplicates → non-idempotent init; uploads dying on navigation → running in UI scope, not WorkManager.
- **Privacy:** don't log file contents/URIs that reveal PII; log sizes/types.

17.16 System Design Interview Preparation

> ≥25 questions.

Fundamentals

1. *Why are mobile uploads hard?* Large files, flaky networks, backgrounding/ process death, memory limits, duplicate risk.
2. *Multipart vs resumable?* Multipart = one request (restart on fail); resumable = chunks + resume from offset (large/flaky).
3. *Why stream the body?* Avoid OOM — O(chunk) memory, not O(file).
4. *Why run uploads in WorkManager?* Survive navigation and process death; constraints + backoff.
5. *What makes resume possible?* A durable offset + server `status` / `Content-Range` so you continue from the last received byte.

Architecture

6. *Where does the transfer run?* A WorkManager worker (foreground service for long uploads); the UI only observes progress.
7. *What's persisted?* uploadId + offset + idempotency key (Room).
8. *Who is the source of truth for resume?* The server's last-received offset.

Protocol

9. *Resumable steps?* init (get uploadId) → PUT chunks with `Content-Range` → status on failure → resume → finalize.
10. *Idempotency in uploads?* The *init* (create) step carries a key (Ch 15); chunk PUTs are naturally idempotent by range.
11. *308 Resume Incomplete?* Server ack that it got bytes up to an offset; keep going.

Background/lifecycle

12. *Why a foreground service?* Long, user-visible uploads need it to run reliably and show a notification (Ch 46).

13. *Process death mid-upload?* WorkManager reschedules; resume from persisted/ server offset.
14. *Navigate away?* Upload continues in the worker.

Resource/constraints

15. *Constraints?* Unmetered/charging for large uploads (Ch 30/47).
16. *Chunk size trade-off?* Small = overhead; big = more re-send on failure; adapt to network.
17. *Avoid duplicate uploads?* `enqueueUniqueWork(KEEP)` + idempotent init.

Security

18. *Server validation?* Type/size/content, malware scan, re-encode; never trust client MIME.
19. *Signed URLs?* Short-lived, scoped; direct-to-storage offloads bandwidth.
20. *Sensitive media?* Strip EXIF/GPS; consider client-side encryption.

Testing

21. *How test resume?* MockWebServer scripts init/fail/status/finish; assert continues from offset.
22. *How test the worker?* `TestListenableWorkerBuilder` / `WorkManagerTestInitHelper`.
23. *How test no-OOM?* Stream a large file; assert memory bounded (contentLength without buffering).

Production debugging

24. *Large uploads never complete on flaky networks?* Not resumable — switch to chunked.
25. *OOM on upload?* Buffering whole file — stream it.
26. *Duplicate resources?* Non-idempotent init.

Design-change challenges

1. **200 MB video on a flaky connection.** Resumable chunks + WorkManager foreground + durable offset + idempotent init.
2. **Upload must survive app kill.** WorkManager; resume from server offset.
3. **Upload only on Wi-Fi + charging.** WorkManager constraints.
4. **Pause/resume.** Cancel worker, persist offset; re-enqueue to resume from offset.
5. **Scale to millions of uploads.** Signed-URL direct-to-object-storage + backend post-processing.

Mock walkthrough (abridged)

Clarify (size, reliability, background, dup risk) → multipart vs resumable by size → stream the body → WorkManager foreground worker + constraints + backoff → resumable protocol (init/chunk/status) with durable offset → idempotent init → progress via WorkInfo → security (server validation) → testing.

17.17 Practical Exercises

1. **Beginner.** Stream a small multipart upload with progress (no whole-file in memory). *Eval:* bounded memory; progress reported.
2. **Intermediate.** Move it into a WorkManager worker with retry/backoff + Wi-Fi constraint; observe progress via `WorkInfo`. *Eval:* survives navigation; retries.

3. **Advanced.** Implement resumable chunked upload with a persisted offset and server-status resume; prove a mid-upload failure resumes (not restarts). *Eval:* low bytes re-sent.
4. **Production debugging.** Fix an upload that OOMs (buffering), dies on navigation (UI scope), and duplicates on retry (no idempotent init). *Eval:* streamed, backgrounded, idempotent.
5. **System design challenge.** Add pause/resume and adaptive chunk sizing by network quality. *Eval:* resumes from offset; chunk size adapts.

(Solutions withheld unless requested.)

17.18 Chapter Summary & Reference Sheet

Key decisions: choose strategy by **size** — streamed **multipart** for small files, **resumable chunked** for large; **stream the body** (O(chunk) memory, never buffer whole file); run transfers in a **WorkManager foreground worker** so they survive navigation and process death, with **constraints** and **backoff**; persist **uploadId + offset** and **resume from the server's last-received byte**; make the **init step idempotent** (Ch 15) and use **unique work** to prevent duplicates; the UI only **observes progress**, it doesn't run the upload.

Concepts: streamed vs buffered body, multipart vs resumable/chunked, `Content-Range`/offset resume, WorkManager + foreground service, durable progress, idempotent create, network constraints.

Common mistakes: buffering the whole file (OOM); running uploads in UI/ ViewModel scope (die on navigation/death); whole-file restart on failure (never completes); non-idempotent init (duplicates); trusting client MIME/size; no unique work (duplicate transfers).

Checklist:

- [ ] stream body
- [ ] multipart vs resumable by size
- [ ] WorkManager foreground worker
- [ ] constraints + backoff
- [ ] durable offset + server-status resume
- [ ] idempotent init + unique work
- [ ] progress via `WorkInfo`
- [ ] cancel/pause
- [ ] server validates
- [ ] no PII in logs.

Key APIs: `RequestBody`/`BufferedSink` (streamed), `MultipartBody.Part`, `@Multipart`/`@Part`, `Content-Range`, `CoroutineWorker`/`@HiltWorker`, `setForeground`/`ForegroundInfo`, `Constraints`/`NetworkType`, `enqueueUniqueWork`, `setProgress`/`WorkInfo`, `ContentResolver.openInputStream`.

References:

- OkHttp request bodies / multipart: https://square.github.io/okhttp/recipes/#posting-a-multipart-request-java
- WorkManager: https://developer.android.com/topic/libraries/architecture/workmanager
- Long-running / foreground workers: https://developer.android.com/topic/libraries/architecture/workmanager/advanced/long-running

- Resumable upload protocol (tus): https://tus.io/protocols/resumable-upload
- Google resumable uploads (reference): https://cloud.google.com/storage/docs/performing-resumable-uploads

Chapter review (Phase 4)

- **Completeness:** all 18 sections; architecture/class/sequence/failure diagrams; streamed body + resumable worker + enqueue + multipart + progress; concurrency (unique work, server-offset truth); failure/resume matrix; 5 levels; multipart- vs-resumable and via-backend-vs-direct trade-offs; security (server validation); perf (streaming, chunk size); MockWebServer + worker tests; observability; 26 Q&A
    - 5 challenges. ✅
- **Expand later:** WorkManager depth (Ch 30), foreground services (Ch 46), SAF/ content providers (Ch 62), media processing, download manager (Ch 18).
- **Interview coverage:** exceeds ≥25 bar.